

ALB'DO + FORGE

A Rust-native render compiler and a backend it forges from your code.

Low-level performance, high-level simplicity: write ordinary React/TSX;
let the compiler do the optimization a human never would by hand — and
let it forge the database instead of asking you to wire one up.

01

The product in one breath

*Every framework asks “how do I connect my frontend to my backend?” ALB'DO's answer: **there is no seam, because the compiler never split them into two systems.** You write your app. It emits the interface. It forges the database.*

ALB'DO is a Rust-native render compiler and HTTP runtime for JSX/TSX. A developer writes ordinary React. The compiler reads each component's effects at build time and classifies it into a **hydration tier** — and that single decision is the whole game. There is nothing to configure.

FORGE is the same refusal, one layer deeper. Instead of making you stand up, wire, and reconcile a second system for your data, the compiler infers persistence from the code itself and forges the storage as a build artifact. “The database is a tier the compiler emits.” Together they are a complete full-stack ecosystem from a single program and a single binary — no Node.js in the request path.

The tier decision

Tier	When the compiler picks it	What ships
A	No hooks, no async, no side effects — pure UI	Plain HTML, zero JavaScript , served from a precomputed frame. Sub-millisecond GET lives here.
B	Event handlers, light server-driven interactivity	Only that island ships JS; minimal patches stream over the wire.
C	Full hooks / async / rich side effects	Hydrates and runs on the client — instant and local, exactly like React.

On top of that, a **binding-mode** path takes many stateful components and ships a few hundred bytes of state bindings instead of hydrating the whole component: the server-rendered DOM stays put and a click updates the bound node locally, with no VDOM, no re-render, and no network round-trip.

02

The thesis — why this is worth building

The novelty is that the **compiler** makes the placement call a human never would by hand: it lowers everything to a Struct-of-Arrays IR and streams DOM mutations as a binary opcode program. No mainstream framework does that optimization for you.

FORGE extends the exact same machinery to data. The engine already turns a `useState` call into a slot that is written on the server and hydrated on the client through a signal graph. The whole paradigm collapses to one observation:

***Persistent, shared state is just a slot that outlives the request.** A value the compiler can prove escapes request scope and is shared across sessions is, by definition, a database row. The compiler's job is to notice that boundary crossing — and forge storage for it.*

The tierless dream, on a surface people actually use

One program, compiler-sliced across client, server, and database, with all the boilerplate (connections, queries, serialization, the wire protocol) generated — this is the multitier programming idea that Ur/Web, Links, Eliom, and ScalaLocs pursued for two decades. Every one of them invented a *new language nobody wanted to learn*. That is the entire opening: ALB'DO does tierless slicing in the **React/TSX ten million developers already write**, and it is *already* a tier-slicing compiler. The idea isn't being built from zero — it's one compiler pass away from a target the machinery already reaches.

Why the synchronous JS runtime is a feature, not a bug

Server-side JS runs inside QuickJS, which is synchronous and has no I/O — a handler cannot await a database. Every other framework would fight this. Because the compiler knows the full data graph ahead of time, **Rust pre-resolves all data before entering QuickJS**. The render stays pure and synchronous; fetching happens in the fast native layer; the JS never needs async. The constraint that would shatter a bolt-on data layer is exactly what forces the correct compile-time architecture.

03

Feature surface — ALB'DO (the frontend, shipped)

The critical path is complete and live-verified against a real flagship app, **Halation** (a literary journal). Everything below works today in both `albedo dev` and `albedo serve`, which share one rendering pipeline.

Area	Capability
Real JS, both ends	SWC-parsed <code>.tsx/jsx</code> rendered under an embedded QuickJS engine on SSR + actions; real loops, <code>try</code> , array methods. A broken construct fails loudly with an overlay error instead of rendering null.
Client hydration	<code>useState</code> / <code>useEffect</code> / <code>useRef</code> / <code>useMemo</code> / <code>useCallback</code> / <code>useContext</code> islands hydrate in the browser and run client-side with no round-trip.
React/Next compat	File-based routing with nested <code>layout.tsx</code> ; dynamic <code>/essays/[slug]</code> routes with params; error/loading boundaries; <code><Link></code> soft-nav + prefetch.
Data-in forms	Server <code>action()</code> handlers with zod validation; compile-time-anchored error projection onto form slots.
Metadata & styling	Dynamic <code><title></code> / <code>meta</code> incl. <code>generateMetadata()</code> ; Global CSS + CSS Modules; app-wide theming with pre-paint (no FOUC).
npm deps	Third-party packages bundled in-tree for SSR + actions (tested against zod, date-fns).
Dev experience	<code>albedo init</code> scaffold; dev server with hot reload over SSE, running the <i>production</i> pipeline plus watch + hot-swap — no second renderer that can rot.
Honest perf	An end-to-end HTTP harness fires real GET TTFB and POST action p50/p99 (cold vs warm) plus in-process Criterion benchmarks. Numbers are measured, published, and reproducible — never claimed.

04

Feature surface — FORGE (the backend-less backend)

FORGE is organized as six pillars, each a real research result crossed with a real seam in the engine. The first slice is **live and tested**; the rest is a sequenced roadmap (§08).

#	Pillar	What it does
1	Persistence by escape analysis	A compiler pass over the component graph infers which values outlive the request and are shared across sessions → the inferred schema. Zero markers: no schema, no model, no db object.
2	Data is tier-sliced	The same tier decision that places a component places each datum: baked at build (Tier A), fetched per-request (Tier B), or synced to the client (Tier C).
3	Automatic schema + migrations	The compiler diffs the inferred schema between builds and emits the migration as a build artifact. The developer never writes a migration.
4	Query synthesis, content-addressed	Because the compiler sees every read across the whole graph, it synthesizes provably-safe, N+1-free queries and batches them to the route boundary. Each query's structural hash is its cache key and IVM node id.
5	Incremental reactivity (IVM)	Storage deltas propagate through the same signal graph that re-renders islands — recompute the delta, never the whole query — out over the wire, painted into the DOM by Phosphor. Row change and UI re-render are one dataflow graph.

#	Pillar	What it does
6	Durable writes	The server <code>action()</code> mutation entry point is upgraded into a durable, exactly-once workflow: the checkpoint rides inside the write's transaction, so a crash mid-action rolls back cleanly and resumes.

The closer: the data layer learns the app (CTRNI'TAS)

Escape analysis gives the *initial* placement. The runtime then observes real behaviour — hot rows, read/write ratios, which synthesized queries fire, tier mispredictions — and feeds it back so the compiler **auto-creates indexes** for the reads that happen, **auto-materializes** hot queries as incremental views, and **re-replaces** data across tiers. A database can auto-index but can't see your components; a framework can cache but doesn't own storage. ALB'DO owns both ends of the compile→serve loop — the only system that can close the telemetry loop over *data*. That is the uncopyable capability.

05

The codebase — a technical view

A single Cargo workspace. The library crate `dom-render-compiler` (`src/`, ~800-line `lib.rs` facade) holds the compiler + render kernel; `crates/albedo-server` is the axum + tokio HTTP runtime and the `albedo` binary; `crates/albedo-node` holds cross-platform bindings.

Module map (`src/`)

Path	Responsibility
<code>effects.rs</code>	Effect analysis → the Tier A/B/C decision (<code>decide_tier_and_hydration()</code>). The heart of the whole system.
<code>parser.rs</code>	SWC-based JSX/TSX parser.
<code>graph.rs</code>	<code>ComponentGraph</code> + dependency extraction — the seam FORGE escape analysis reads.
<code>ir/</code>	Struct-of-Arrays columnar IR + SIMD dirty-scan; the opcode wire representation.
<code>manifest/</code>	Build manifest (<code>RenderManifestV2</code>), shell composition, and the <code>DataDep / DataSource</code> schema carrier.
<code>bundler/</code>	<code>classify</code> → <code>plan</code> → <code>rewrite</code> → <code>emit</code> pipeline; npm bundling.
<code>hydration/</code> , <code>transforms/</code>	Island hydration planning and JSX transforms.
<code>routing/</code>	File-based + dynamic route resolution.
<code>runtime/</code>	Render kernel, 4-lane scheduler, request-scoped QuickJS engine + arena, <code>SlotStore</code> , <code>BroadcastRegistry</code> .
<code>forge/</code>	The FORGE storage plane (detailed below).

The four hot paths (where the engineering obsession goes)

- **HP1 — Render** → **wire encode**: IR columns → opcodes → bytes, branchless, no-alloc-per-tick (asserted by an allocation-counter test).
- **HP2 — The serve / I/O loop**: request in → response bytes out; the render core is transport-agnostic (`build_stream` returns a `Stream<Bytes>`; axum only wraps it).
- **HP3 — JS execution**: QuickJS server-side under a request-scoped bump arena (the unsafe $O(1)$ reset was removed after a cross-request use-after-free; do not reintroduce it).
- **HP4 — The compile loop**: edit → rebuilt artifacts; incremental cache on the dev-watch path today.

06

FORGE internals — what exists in code today

The runtime storage plane lives in `src/forge/` (~700 LOC), gated behind a `forge` Cargo feature so `main` is untouched. The design deliberately mirrors the engine's existing `DepLoyAdapter` pattern: one pluggable trait, neutral value types, storage crates never leaking into the compiler.

File	Role
<code>substrate.rs</code>	The <code>DataSubstrate</code> trait — FORGE's lower boundary. Object-safe, async: <code>migrate()</code> , <code>query()</code> , <code>execute()</code> . Every backend (<code>libSQL</code> now, <code>BYO-Postgres</code> and <code>edge-KV</code> later) implements this one seam.

File	Role
value.rs	Substrate-neutral SqlValue / Row / Rows / SubstrateError. The engine never names a storage crate.
libsql.rs	LibSqlSubstrate — the Phase 0 backend that actually executes SQL (local file or :memory:). Live and tested.
mem.rs	RecordingSubstrate — an in-memory test double to exercise wiring before a real backend is attached.
skeleton.rs	Gate-1 walking skeleton: a hand-authored topic → query map (a guestbook) standing in for what escape analysis will infer. Bootstraps + seeds the table, materialises rows to JSON, seeds the matching BroadcastRegistry topic.

The read loop that works end-to-end today (Gate 1)

Storage → hydrate → render → HTML, proven by tests/forge_gate1_e2e.rs: a real libSQL substrate is bootstrapped and seeded; the FORGE skeleton materialises the guestbook topic into the broadcast registry; a component reads it via useSharedSlot; the SSR HTML comes out carrying the persisted rows — **with no per-request DB I/O and no query authored by hand in the component.**

```
// tests/forge_gate1_e2e.rs (feature = "forge")
let db = LibSqlSubstrate::open_in_memory().await?;
skeleton::bootstrap_schema(&db).await?;          // create + seed table

let broadcast = BroadcastRegistry::new();
skeleton::hydrate_topics(&db, &broadcast).await?; // storage -> topic value

let out = render_entry_with_broadcast(&project, "Component.tsx", ...)?;
assert!(out.html.contains("first light"));        // persisted row is on the page
```

The compile-time half already has its seam too: the manifest carries DataDep { key, source } where DataSource::DbQuery { query, param_keys } is exactly what an inferred read will populate — the serve path pre-resolves those into props before the synchronous QuickJS render. FORGE plugs into a seam the engine already had.

07

Measured performance (honest, reproducible)

One 16-core machine, release build, over the wire on loopback. These say what ALB'DO's own overhead is; they are not a full load test and do not simulate your network or database. "Sub-millisecond" means **server response for page loads (GET) and actions (POST)**; client interactions are instant and local like React.

Request latency — GET / SSR shell (28.8 KB starter page)

Connection model	Conc.	TTFB p50	TTFB p99
keep-alive, uncontended	1	0.07 ms (70 µs)	0.17 ms
keep-alive, steady	8	0.13 ms	0.30 ms
keep-alive, all cores	16	0.23 ms	0.53 ms
new connection per request	1	0.36 ms	0.54 ms

Action round-trip — POST /_albedo/action (decode → run → encode)

Connection model	Conc.	TTFB p50	TTFB p99
keep-alive, uncontended	1	0.24 ms	0.43 ms
keep-alive, all cores	16	0.45 ms	1.34 ms
new connection per request	1	0.50 ms	1.21 ms

In-process cost & footprint

What	Time / size
Server action dispatch (decode → run → encode, no socket)	~13.6 µs
Static (Tier A) route — framework shell, no client JS	~315 B
One interactive island (handler wrapper + bindings)	~250–400 B
Cold process start — spawn to first render	~0.5 s; first render 0.67 ms → 0.11 ms warm
Clean build — 5-component starter	~434 ms

08

Where it stands, and where it goes

The body is shipped. Tiering, the SoA IR + SIMD dirty-scan, the opcode wire, the 4-lane runtime, and the request-scoped QuickJS arena are real, proven, and exercised by the flagship. FORGE's Phase 0 read loop is live and feature-gated off the mainline. **The soul is the active work** — the from-scratch low-level craft that turns "a fast framework" into "the thing only this person would have built."

Stage	Theme	Ships	Paywall
1	Adoption	Deploy manifold (adapter architecture: static → Vercel → container) + codebase readiness sweep. FORGE Phase 0 rides here as a teaser.	None — free everywhere

Stage	Theme	Ships	Paywall
2	Soul as moat	Self-optimizing loop prototype · live p99 tail-latency demo · columnar wire codec size number. FORGE Phase 4 (CTRNI'TAS over data) rides here.	Pro (hosted self-opt + AI)
3	The cloud	The ALB'DO cloud (cheapest per-request compute, optimal tier placement) · edge adapter if the WASM spike passes. Adapters stay open.	Cloud / Enterprise

FORGE's five phases

- **Phase 0 — the spike (live):** one embedded substrate, one persistent collection, pre-resolved into props, one durable write. The “wait, where’s the backend?” moment.
- **Phase 1 — inference core:** full escape analysis → inferred schema; auto-migration; content-addressed query synthesis; data folded into the tier decision.
- **Phase 2 — reactive IVM:** DBSP-style incremental view maintenance; deltas ride the broadcast registry + wire codec; Phosphor paints reactive queries server→screen.
- **Phase 3 — pluggable substrates + durable writes at scale:** Postgres and edge-KV targets; exactly-once crash-resumable actions against a transactional store.
- **Phase 4 — the closed loop (CTRNI'TAS over data):** telemetry → auto-index, auto-materialize, re-place, and train the data-delta wire. The uncopyable, hosted-premium capability.

09

Positioning — brand-separate, system-fused

ALB'DO and FORGE split cleanly along *what kind of data an app has*, but they are one fused compiler — the invention exists *only because one compiler owns both ends* (whole-graph query synthesis, pre-resolution before QuickJS, one telemetry loop). FORGE is a product name and a price drawn around part of what the single compiler emits, never a second system talking over a wire.

Tier	Name	What FORGE adds on top of the free frontend
Free	Sol	The entire “no backend” magic on your own metal — escape-analysis persistence, inferred schema, auto-migrations, query synthesis, durable actions, embedded substrate <i>and</i> BYO-Postgres. Deploy anywhere. Never gated.
Pro	Equinox	Hosted FORGE: managed substrate + the self-optimizing data loop (“your database tunes itself while you sleep”) + graphify-grounded AI over your data.
Ultra	Umbra	Priority heavy optimization (full IVM materialization + trained data-delta wire on dedicated compute), cross-region sync, team seats/RBAC.
Enterprise	Persephone	Sovereign / on-prem managed data, data-sovereignty, compliance, dedicated isolation.

*The governing line: the **entire framework is given away whole**; every paid tier sells only compute that runs on our machines. The local capability is never padlocked — no crippleware, no fork incentive. The premium is hosted, therefore uncopyable.*

ALB'DO refuses to make the developer choose where their UI runs — the compiler decides, better than a human would. FORGE is the same refusal, one layer deeper: stop making the developer stand up, wire up, and reconcile a second system. Infer it. Forge it. Tune it to the app. Then there is no backend — there is only the program, and the compiler that knows it whole.

Prototype · v0.1.0 · built by Bishal Sen · github.com/Sen-Bishal · 2026. This document reflects the codebase and roadmap as of the FORGE branch; APIs are still moving.